

—Assignment 3—

Ques 1:

Explain difference between:

- **GET**
- **POST**
- **PUT**
- **DELETE**

Why is FastAPI preferred over Flask for modern APIs?

Answer:

APIs are very important in Machine Learning because trained models are usually deployed as APIs so that applications can send data and receive predictions. HTTP methods such as GET, POST, PUT, and DELETE are used to communicate with these ML services.

1. GET

The GET method is used to retrieve information from a server without changing any data.

Example in ML:

GET /model/status

This request can be used to check whether a deployed fraud detection model is running successfully.

2. POST

The POST method is used to send new data to the server.

Example in ML:

POST /predict

A customer transaction is sent to the fraud detection model, and the model returns a prediction.

Input:

```
{  
  "amount": 5000,  
  "location": "Delhi"  
}
```

Output:

```
{  
  "fraud_probability": 0.12  
}
```

POST is the most commonly used method in Machine Learning APIs.

3. PUT

PUT is used to update existing resources.

Example in ML:

PUT /model/config

This can update model parameters, thresholds, or deployment settings.

4. DELETE

DELETE is used to remove resources from the server.

Example in ML:

DELETE /model/v1

This request removes an outdated model version from production.

Comparison Table

Method	Purpose	ML Example
GET	Retrieve information	Check model status
POST	Send data for prediction	Fraud prediction request
PUT	Update existing resource	Update model configuration
DELETE	Remove resource	Delete old model version

Why is FastAPI Preferred Over Flask for Modern APIs?

FastAPI has become very popular in Machine Learning deployment because it provides better performance and several built-in features.

i)- High Performance

FastAPI is faster than Flask because it supports asynchronous processing. This is important when thousands of prediction requests are received every second.

ii)- Automatic API Documentation

FastAPI automatically generates Swagger documentation.

This helps ML engineers test endpoints such as:

`/predict`

`/train`

`/model/status`

without writing extra code.

iii)- Built-in Data Validation

Machine Learning models require correct input formats.

FastAPI validates incoming data automatically.

Example:

```
class Transaction(BaseModel):
```

```
    amount: float
```

```
    age: int
```

If incorrect data is sent, FastAPI immediately returns an error.

iv)- Better Type Hint Support

FastAPI uses Python type hints extensively, making code easier to read and maintain.

v)- Widely Used in ML Deployment

FastAPI is commonly used for:

- Model serving
 - Prediction APIs
 - Recommendation systems
 - Fraud detection systems
 - NLP applications
-

Question 2:

Create a FastAPI application with:

- Root endpoint (/)
- /courses GET endpoint
- /courses/{course_id} using path parameter

Add:

- Query parameter level (optional)

Explain difference between path parameter and query parameter.

Answer:

FastAPI is a modern Python framework used to build APIs quickly and efficiently. It allows developers to create endpoints, handle user requests, validate data, and serve Machine Learning models. In this example, we create a simple FastAPI application with a root endpoint, a courses

endpoint, and a course details endpoint that uses both path and query parameters.

```
from fastapi import FastAPI

app = FastAPI()

# Root Endpoint
@app.get("/")
def home():
    return {"message": "Welcome to PW Skills API"}

#Endpoints
@app.get("/courses")
def get_courses():
    return {
        "courses": [
            "Python",
            "Machine Learning",
            "Data Science",
            "Deep Learning"
        ]
    }

# Endpoints
@app.get("/courses/{course_id}")
def get_course(course_id: int, level: str = None):
    return {
        "course_id": course_id,
        "level": level
    }
```

```
}
```

Example Requests

Root Endpoint

GET /

Output:

```
{  
  "message": "Welcome to PW Skills API"  
}
```

Courses Endpoint

GET /courses

Output:

```
{  
  "courses": [  
    "Python",  
    "Machine Learning",  
    "Data Science",  
    "Deep Learning"  
  ]  
}
```

Path Parameter Example

GET /courses/101

Output:

```
{  
  "course_id": 101,  
  "level": null  
}
```

Query Parameter Example

GET /courses/101?level=beginner

Output:

```
{  
  "course_id": 101,  
  "level": "beginner"  
}
```

Difference Between Path Parameter and Query Parameter

Path Parameter

A path parameter is part of the URL path and is generally used to identify a specific resource.

Example:

/courses/101

Here:

101

is the path parameter.

It tells the API which specific course is being requested.

Query Parameter

A query parameter is added after the ? symbol in the URL and is used to filter, sort, or provide additional information.

Example:

`/courses/101?level=beginner`

Here:

`level=beginner`

is the query parameter.

It provides additional details about the request.

Comparison Table

Feature	Path Parameter	Query Parameter
Location	Inside URL path	After ? in URL
Purpose	Identify a resource	Filter or modify results

Mandatory	Usually required	Usually optional
Example	<code>/courses/101</code>	<code>?level=beginner</code>

Real-World ML Example

Suppose a fraud detection API is deployed:

Path Parameter

`/predictions/1001`

Retrieves prediction details for transaction **1001**.

Query Parameter

`/predictions/1001?details=true`

Requests additional information about the prediction.

Question 3:

Add validation to:

- `course_id` must be integer > 0
- `level` must be one of: beginner, intermediate, advanced

What happens if validation fails?

Answer:

Data validation is an important feature in FastAPI. It ensures that users send correct data to the API. Validation helps prevent errors, improves security, and ensures that the application receives data in the expected format. FastAPI automatically performs validation using Python type hints and Pydantic.

In this example, we want to ensure that:

- `course_id` is an integer greater than 0.
- `level` can only be "beginner", "intermediate", or "advanced".
-

CODE:

```
from fastapi import FastAPI, Path, Query
from typing import Optional

app = FastAPI()

@app.get("/courses/{course_id}")
def get_course(
    course_id: int = Path(..., gt=0),
    level: Optional[str] = Query(
        None,
        pattern="^(beginner|intermediate|advanced)$"
    )
):
    return {
```

```
"course_id": course_id,  
"level": level  
}
```

Valid Request Example

GET /courses/101?level=beginner

Output:

```
{  
  "course_id": 101,  
  "level": "beginner"  
}
```

Validation Rule 1: $\text{course_id} > 0$

Invalid Request

GET /courses/-5

Since `course_id` is less than 0, FastAPI rejects the request.

Output:

```
{  
  "detail": [  
    {  
      "msg": "Input should be greater than 0"  
    }  
  ]  
}
```

Validation Rule 2: Allowed Values for level

Invalid Request

`GET /courses/101?level=expert`

The value `"expert"` is not allowed because only:

beginner
intermediate
advanced

are valid options.

Output:

```
{  
  "detail": [  
    {  
      "msg": "String should match pattern"  
    }  
  ]  
}
```

What Happens If Validation Fails?

When validation fails, FastAPI:

1. Stops Processing the Request

The API endpoint function is not executed.

2. Returns an Error Response

FastAPI automatically returns:

HTTP Status Code: 422

(422 Unprocessable Entity)

3. Provides Error Details

The response clearly explains:

- Which field caused the error.
- Why validation failed.
- What value was expected.

Example

GET /courses/0?level=expert

Possible Response:

```
{
  "detail": [
    {
      "loc": ["path", "course_id"],
      "msg": "Input should be greater than 0"
    },
    {
      "loc": ["query", "level"],
      "msg": "String should match pattern"
    }
  ]
}
```

Benefits of Validation

- Prevents incorrect data entry.
- Improves API reliability.
- Reduces application errors.
- Enhances security.
- Saves developer debugging time.

In Machine Learning APIs, validation is especially important because models require correctly formatted input data to generate accurate predictions.

Question 4: Create a Pydantic model:

```
class Course(BaseModel):  
    title: str  
    price: float  
    duration: int  
    published: bool
```

Add:

- **Minimum price validation**
- **Duration > 0**
- **Default published = False**

Explain why schema validation is critical in APIs.

Answer:

Pydantic is a data validation library used by FastAPI to validate incoming request data automatically. It ensures that data sent by users follows the required structure and data types before it reaches the application logic. This helps developers build reliable and secure APIs.

In this example, we need to add validation rules to ensure that the course price is positive, the course duration is greater than zero, and the published field has a default value of False.

Pydantic Model with Validation

```
from pydantic import BaseModel, Field
```

```
class Course(BaseModel):  
    title: str  
    price: float = Field(..., gt=0)  
    duration: int = Field(..., gt=0)  
    published: bool = False
```

Explanation of Validation Rules

1. Minimum Price Validation

```
price: float = Field(..., gt=0)
```

Here:

- **gt** means "greater than".
- The price must be greater than 0.
-

Valid Example:

```
{  
  "title": "Machine Learning",  
  "price": 4999,  
  "duration": 30  
}
```

Invalid Example:

```
{  
  "title": "Machine Learning",  
  "price": -100,  
  "duration": 30  
}
```

FastAPI will return a validation error.

2. Duration Greater Than Zero

```
duration: int = Field(..., gt=0)
```

This ensures that course duration cannot be zero or negative.

Valid Example:

```
{  
  "duration": 45  
}
```

Invalid Example:

```
{  
  "duration": -10  
}
```

Such data is rejected automatically.

3. Default Published Value

`published: bool = False`

If the user does not provide the published field, FastAPI automatically assigns:

False

Example Input:

```
{  
  "title": "Data Science",  
  "price": 2999,  
  "duration": 20  
}
```

Stored Value:

```
{  
  "title": "Data Science",  
  "price": 2999,  
  "duration": 20,  
  "published": false  
}
```

What Happens If Validation Fails?

Suppose a user sends:

```
{  
  "title": "Python",  
  "price": -500,  
  "duration": 0  
}
```

```
}
```

FastAPI automatically returns:

```
{  
  "detail": [  
    {  
      "msg": "Input should be greater than 0"  
    }  
  ]  
}
```

along with:

HTTP Status Code: 422 Unprocessable Entity

The API endpoint will not execute until valid data is provided.

Why is Schema Validation Critical in APIs?

Schema validation is one of the most important features of modern APIs because it ensures that only correct and meaningful data enters the system.

1. Prevents Invalid Data

Without validation, users might send:

```
{  
  "price": -1000,  
  "duration": -5  
}
```

which makes no practical sense.

Validation prevents such incorrect data.

2. Improves Data Quality

Validated data ensures that databases contain accurate and meaningful information.

This improves:

- Reporting
- Analytics
- Machine Learning model performance

3. Enhances Security

Validation protects APIs from malicious or unexpected inputs.

It helps reduce:

- Invalid requests
- Injection attacks
- System crashes

4. Reduces Application Errors

When incorrect data is rejected early, developers spend less time debugging issues later.

This increases system reliability.

5. Better User Experience

Instead of application failures, users receive clear error messages explaining what is wrong with their input.

Example:

Price must be greater than 0.

This makes APIs easier to use.

Question 5:

Explain:

- **Authentication vs Authorization**
- **JWT-based authentication**
- **Why stateless authentication is scalable**

Answer:

Security is one of the most important aspects of modern APIs and web applications. Organizations need to ensure that only legitimate users can access their systems and that users can perform only those actions for which they have permission. This is achieved through **Authentication** and **Authorization**. Modern applications commonly use **JWT (JSON Web Token)** for secure and scalable authentication.

1. Authentication vs Authorization

Although Authentication and Authorization are closely related, they serve different purposes.

Authentication

Authentication is the process of verifying the identity of a user.

In simple words, it answers the question:

"Who are you?"

When a user logs into an application using a username and password, the system checks whether the provided credentials are correct.

Example

Suppose a student logs into a learning platform using:

Username: Rahul123

Password: *****

The system verifies the credentials.

If they are correct, the user is authenticated.

Common Authentication Methods

- Username and Password
- OTP (One-Time Password)
- Biometric Authentication
- JWT Tokens
- OAuth Login (Google, GitHub, Facebook)

Authorization

Authorization happens after authentication.

It determines what an authenticated user is allowed to do.

It answers the question:

"What are you allowed to access?"

Example

In an online learning platform:

- Student can view courses.
- Instructor can upload lectures.
- Admin can manage users.

All users may be authenticated, but they have different permissions.

This permission control is called authorization.

.....

2. JWT-Based Authentication

JWT stands for **JSON Web Token**.

It is a secure method of transmitting user information between a client and a server.

After a successful login, the server generates a token and sends it to the user.

The user includes this token in future requests.

JWT Workflow

Step 1: User Login

The user sends credentials:

```
{
```

```
"username": "rahul",
```

```
"password": "password123"
```

```
}
```

Step 2: Server Verifies Credentials

If the credentials are correct, the server generates a JWT.

Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Step 3: Token Sent to Client

The server returns:

```
{
```

```
"access_token": "JWT_TOKEN"
```

```
}
```

Step 4: Client Uses Token

For future requests:

```
Authorization: Bearer JWT_TOKEN
```

Step 5: Server Validates Token

The server checks:

- Token signature
- Expiration time

- User information

If valid, access is granted.

Advantages of JWT Authentication

- Fast authentication.
- No session storage required.
- Easy integration with APIs.
- Works well with microservices.
- Supports distributed systems.

.....

3. Why Stateless Authentication is Scalable

Traditional authentication systems often use server-side sessions.

Session-Based Authentication

Workflow:

User Login



Server Creates Session



Session Stored in Memory



Every Request Uses Session ID

Problem:

As the number of users increases, the server must store and manage thousands of sessions.

This increases:

- Memory usage
 - Infrastructure cost
 - System complexity
-

Stateless Authentication

JWT authentication is stateless.

Workflow:

User Login



JWT Generated



Client Stores Token



Client Sends Token with Requests



Server Verifies Token

The server does not store user sessions.

All required information is contained inside the token itself.

Why It Is More Scalable

1. No Session Storage

The server does not maintain user sessions.

Benefits:

- Reduced memory consumption.
 - Better resource utilization.
-

2. Easy Horizontal Scaling

Multiple servers can validate the same token.

Example:

Server 1

Server 2

Server 3

All servers can process requests independently.

No shared session database is required.

3. Better Performance

Since session lookups are not required:

- Request processing becomes faster.
 - Response times improve.
-

4. Suitable for Microservices

Modern applications often use multiple services.

JWT allows each service to independently verify user identity.

This simplifies architecture and improves scalability.

Real-World Example

Consider Netflix.

Millions of users access the platform simultaneously.

Using session-based authentication would require managing millions of active sessions.

Instead, JWT-based authentication allows:

- Fast authentication.
- Distributed processing.
- Better scalability.

This enables the platform to handle large numbers of users efficiently.

Question 6:

Why is API testing critical?

Difference between:

- **Unit testing**
- **Integration testing**

Write a sample test case using FastAPI TestClient.

Answer:

API testing is an important part of software development because APIs act as the communication bridge between applications, databases, machine learning models, and users. If an API fails, the entire application may stop functioning correctly. Therefore, testing ensures that APIs work as expected, return correct responses, handle errors properly, and maintain reliability.

Why is API Testing Critical?

API testing helps verify that an API performs correctly under different conditions.

Some important reasons are:

1. Ensures Correct Functionality

Testing verifies that endpoints return the expected output.

Example:

GET /courses

should return the list of available courses.

2. Detects Bugs Early

API testing helps identify issues before the application reaches users.

Benefits:

- Reduced debugging effort
 - Faster development
 - Better software quality
-

3. Improves Reliability

Regular testing ensures APIs continue working correctly after updates or changes.

This prevents unexpected failures in production.

4. Validates Security

Testing helps ensure:

- Authentication works properly.
 - Unauthorized users cannot access protected resources.
 - Sensitive data remains secure.
-

5. Supports ML and Data Science Applications

In Machine Learning systems, models are often deployed through APIs.

Testing ensures:

- Predictions are generated correctly.
 - Input validation works properly.
 - Model endpoints remain available.
-

Unit Testing

Unit Testing focuses on testing a single component or function independently.

The goal is to verify that an individual piece of code behaves correctly.

Characteristics

- Tests one function at a time.
- Fast execution.
- Easy to identify bugs.
- Does not depend on external systems.

Example

Testing a function:

```
def add(a, b):  
    return a + b
```

Unit test checks whether:

```
add(2, 3)
```

returns:

5

Integration Testing

Integration Testing checks whether multiple components work together correctly.

Instead of testing a single function, it tests the interaction between modules.

Characteristics

- Tests complete workflows.
- Verifies communication between components.
- May involve databases and APIs.
- More comprehensive than unit testing.

Example

Testing:

Client → API → Database

The test verifies that all components communicate correctly.

.....

Sample FastAPI Application

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def home():

    return {"message": "Welcome to FastAPI"}
```

Sample Test Case Using FastAPI TestClient

FastAPI provides **TestClient** for API testing.

Test Code

```
from fastapi.testclient import TestClient

from main import app

client = TestClient(app)

def test_home():

    response = client.get("/")

    assert response.status_code == 200

    assert response.json() == {

        "message": "Welcome to FastAPI"

    }
```

Explanation of Test Case

Step 1

Create a TestClient object.

```
client = TestClient(app)
```

This simulates API requests.

Step 2

Send a GET request.

```
response = client.get("/")
```

The request is sent to the root endpoint.

Step 3

Check Status Code.

```
assert response.status_code == 200
```

This verifies that the API responded successfully.

Step 4

Check Response Data.

```
assert response.json() == {  
    "message": "Welcome to FastAPI"  
}
```

This verifies that the API returned the expected output.

Expected Result

If the test passes:

```
Test Passed
```

If the API returns incorrect data or an error:

Test Failed

and the developer can investigate the issue.

Real-World Example

Consider a Machine Learning API:

POST /predict

The API receives customer data and returns a fraud prediction.

API testing ensures:

- Correct input is accepted.
- Invalid data is rejected.
- Predictions are returned successfully.
- Response format remains consistent.

Without testing, incorrect predictions or application failures may occur.

Question 7: Explain use of:

- Optional
- List[str]
- Union
- Annotated

Why does proper typing improve maintainability

Answer:

Python type hints are widely used in FastAPI and modern software development. They help developers specify the expected type of data being passed to functions, APIs, and models. Proper typing makes code easier to understand, reduces errors, and improves maintainability. FastAPI heavily relies on type hints for validation, documentation, and request handling.

1. Optional

`Optional` is used when a value may either contain a specific type or be `None`.

Example

```
from typing import Optional
def get_course(level: Optional[str] = None):
    return level
```

In this example:

- `level` can be a string.
- It can also be `None`.

Valid Inputs

```
get_course("beginner")
get_course(None)
```

Use Case

In FastAPI, optional query parameters are very common.

Example:

```
@app.get("/courses")
def get_courses(level: Optional[str] = None):
    return {"level": level}
```

If the user does not provide a level, the API still works.

2. List[str]

`List[str]` indicates that a variable contains multiple string values.

Example

```
from typing import List
courses: List[str] = [
    "Python",
    "Machine Learning",
    "Data Science"
]
```

Here:

- The variable is a list.
- Every element must be a string.

Use Case

Suppose an API returns available courses:

```
from typing import List
@app.get("/courses")
def get_courses() -> List[str]:
    return [
        "Python",
```

```
"Machine Learning",  
"Deep Learning"  
]
```

This tells developers exactly what data type the API returns.

3. Union

`Union` allows a variable to accept more than one data type.

Example

```
from typing import Union  
def display(value: Union[int, str]):  
    return value
```

Now the function accepts:

```
display(100)  
display("Python")
```

Both are valid.

Use Case

In APIs, sometimes IDs may be numeric or text-based.

Example:

```
course_id: Union[int, str]
```

This means the API can accept either an integer or a string.

4. Annotated

Annotated is used to combine type hints with additional validation information.

It is commonly used in FastAPI.

Example

```
from typing import Annotated
from fastapi import Query
level: Annotated[str, Query(max_length=20)]
```

Here:

- Data type = string
- Maximum length = 20 characters

FastAPI Example

```
from typing import Annotated
from fastapi import FastAPI, Query
app = FastAPI()
@app.get("/courses")
def get_course(
    level: Annotated[str, Query(max_length=20)]
):
    return {"level": level}
```

FastAPI uses this information for validation and automatic documentation.

.....

Why Does Proper Typing Improve Maintainability?

Maintainability means how easily a program can be understood, modified, and maintained in the future.

Proper typing improves maintainability in several ways.

1. Improves Code Readability

When developers read the code, they immediately understand what type of data is expected.

Example:

```
def get_course(course_id: int):
```

It is clear that `course_id` must be an integer.

2. Reduces Errors

Type hints help identify mistakes before execution.

Example:

```
def add(a: int, b: int):  
    return a + b
```

Passing a string accidentally can be detected quickly.

3. Better IDE Support

Editors like VS Code provide:

- Auto-completion
- Error detection
- Better suggestions

This improves developer productivity.

4. Easier Team Collaboration

When multiple developers work on the same project, type hints act as documentation.

Developers can understand functions without reading extensive comments.

5. Automatic Validation in FastAPI

FastAPI uses type hints to validate incoming requests automatically.

Example:

```
course_id: int
```

If the user sends:

```
course_id = "abc"
```

FastAPI returns a validation error automatically.

6. Easier Maintenance of Large Projects

In large ML and API projects containing hundreds of files, proper typing helps developers quickly understand and update code without introducing bugs.

Question 8: Explain the difference between:

- Path parameter
- Query parameter
- Request body

When should each be used?

Answer:

In FastAPI and REST APIs, data can be sent to the server in different ways. The three most common methods are **Path Parameters**, **Query Parameters**, and **Request Bodies**. Each serves a different purpose and is used depending on the type of information being sent. Understanding these concepts is important for designing clean and efficient APIs.

1. Path Parameter

A Path Parameter is a value that is included directly in the URL path. It is usually used to identify a specific resource.

Example

```
@app.get("/courses/{course_id}")
def get_course(course_id: int):
    return {"course_id": course_id}
```

API Request:

`/courses/101`

Here:

`101`

is the path parameter.

The API understands that the user wants information about course 101.

When to Use Path Parameters

Use path parameters when:

- Identifying a specific resource.
- Fetching a particular record.
- Updating a specific item.
- Deleting a specific item.

Real-World Examples

`/users/5`

`/products/101`

`/orders/5001`

Each URL points to a specific resource.

2. Query Parameter

A Query Parameter is added after the `?` symbol in the URL. It is generally used to filter, sort, or customize results.

Example

```
@app.get("/courses")
def get_courses(level: str = None):
    return {"level": level}
```

API Request:

`/courses?level=beginner`

Here:

`level=beginner`

is the query parameter.

When to Use Query Parameters

Use query parameters when:

- Filtering results.
- Searching data.
- Sorting records.
- Pagination.

Real-World Examples

`/products?category=laptop`

`/courses?level=advanced`

`/users?page=2`

These parameters modify the result without changing the resource itself.

3. Request Body

A Request Body is used to send large or structured data to the server, usually in JSON format.

It is commonly used with:

- POST requests
- PUT requests
- PATCH requests

Example

```
from pydantic import BaseModel
class Course(BaseModel):
    title: str
    price: float
@app.post("/courses")
def create_course(course: Course):
    return course
```

Request:

```
{  
  "title": "Machine Learning",  
  "price": 4999  
}
```

The data is sent inside the request body.

When to Use Request Body

Use request bodies when:

- Creating new records.
- Updating existing records.
- Sending complex data structures.
- Uploading large amounts of information.

Real-World Examples

Creating a user:

```
{  
  "name": "Rahul",  
  "email": "rahul@gmail.com"  
}
```

Creating a product:

```
{  
  "name": "Laptop",  
  "price": 55000  
}
```

.....

Real-World ML API Example

Suppose a Fraud Detection API is deployed.

Path Parameter

`/predictions/1001`

Fetches prediction for transaction 1001.

Query Parameter

`/predictions/1001?details=true`

Requests additional prediction details.

Request Body

```
{  
  "amount": 5000,  
  "location": "Delhi",  
  "device": "Mobile"  
}
```

Provides transaction data for fraud prediction.

Question 9: Difference between:

- **BaseModel**
- **Field**
- **validator**
- **root_validator**

Answer:

Pydantic is a powerful Python library used in FastAPI for data validation and data parsing. It helps ensure that incoming data follows the required

structure and rules before it is processed by the application. Four important components of Pydantic are **BaseModel**, **Field**, **validator**, and **root_validator**. Although they work together, each has a different purpose.

1. BaseModel

`BaseModel` is the foundation of every Pydantic model. It is used to define the structure of the data and automatically validates data types.

Example

```
from pydantic import BaseModel
class Course(BaseModel):
    title: str
    price: float
    duration: int
```

In this example:

- `title` must be a string.
- `price` must be a float.
- `duration` must be an integer.

Purpose

- Defines data structure.
- Performs automatic type validation.
- Converts incoming data into Python objects.

Real-World Use

When creating APIs, request bodies are usually defined using `BaseModel`.

2. Field

`Field` is used to add validation rules and metadata to individual fields.

Example

```
from pydantic import BaseModel, Field
class Course(BaseModel):
    title: str
    price: float = Field(gt=0)
    duration: int = Field(gt=0)
```

Here:

- `gt=0` means greater than zero.
- Negative prices or durations are not allowed.

Purpose

- Add validation constraints.
- Set default values.
- Add descriptions and examples.

Example

```
price: float = Field(
    gt=0,
    description="Course price must be positive"
)
```

3. validator

A `validator` is used to validate or modify a specific field before the data is accepted.

Example

```
from pydantic import BaseModel, validator

class Course(BaseModel):
```



```
title: str
```

```
@validator("title")
def check_title(cls, value):
    if len(value) < 3:
        raise ValueError("Title is too short")
    return value
```

Valid Input

```
{
  "title": "Python"
}
```

Invalid Input

```
{
  "title": "Py"
}
```

Output:

Title is too short

Purpose

- Validate a single field.
- Apply custom business rules.
- Clean or transform data.

4. root_validator

A `root_validator` is used when validation depends on multiple fields together.

Example

```
from pydantic import BaseModel, root_validator
```

```

class Course(BaseModel):
    price: float
    discount_price: float

    @root_validator
    def validate_discount(cls, values):
        price = values.get("price")
        discount = values.get("discount_price")

        if discount > price:
            raise ValueError(
                "Discount price cannot exceed actual price"
            )

        return values

```

Valid Input

```

{
  "price": 5000,
  "discount_price": 3000
}

```

Invalid Input

```

{
  "price": 5000,
  "discount_price": 6000
}

```

Output:

Discount price cannot exceed actual price

Purpose

- Validate multiple fields together.

- Check relationships between fields.
- Enforce business logic.

Question 10: Use Pydantic BaseSettings to:

- Store database URL
- Store secret key
- Store debug flag

Why should secrets never be hardcoded?

Answer:

In modern applications, configuration values such as database URLs, secret keys, API keys, and debug settings should not be stored directly inside the source code. Instead, they should be managed through environment variables and configuration files. Pydantic provides **BaseSettings** to handle application settings in a secure and organized manner.

Using BaseSettings makes applications easier to configure, deploy, and maintain across different environments such as development, testing, and production.

Using Pydantic BaseSettings

settings.py

```
from pydantic import BaseSettings
```

```
class Settings(BaseSettings):
```

```
    database_url: str
```

```
    secret_key: str
```

```
    debug: bool = False
```

```
class Config:
```

```
    env_file = ".env"
```

```
settings = Settings()
```

.env File

```
DATABASE_URL=postgresql://user:password@localhost:5432/mydb
```

```
SECRET_KEY=my_super_secret_key
```

```
DEBUG=True
```

Accessing Settings

```
from settings import settings
```

```
print(settings.database_url)
```

```
print(settings.secret_key)
```

```
print(settings.debug)
```

Output:

```
postgresql://user:password@localhost:5432/mydb
```

```
my_super_secret_key
```

```
True
```

Explanation

Database URL

The database URL stores information required to connect to a database.

Example:

```
postgresql://user:password@localhost:5432/mydb
```

It contains:

- Database type
- Username
- Password
- Host
- Port
- Database name

Instead of hardcoding this value, it is loaded from environment variables.

Secret Key

The secret key is used for security-related operations such as:

- JWT token generation
- Password reset tokens
- Session signing
- Authentication

Example:

```
SECRET_KEY=my_super_secret_key
```

This key should remain private.

Debug Flag

The debug flag controls whether the application runs in development mode.

Example:

DEBUG=True

Development Mode:

- Detailed error messages
- Easier debugging

Production Mode:

DEBUG=False

- Improved security
- Better performance

Why Should Secrets Never Be Hardcoded?

Hardcoding means writing sensitive information directly into the source code.

Example:

```
SECRET_KEY = "my_secret_key_123"  
DATABASE_PASSWORD = "admin123"
```

This is considered a bad security practice.

1. Security Risks

If the source code is uploaded to GitHub or shared accidentally, attackers can immediately access sensitive credentials.

Example:

```
SECRET_KEY = "my_secret_key_123"
```

Anyone viewing the code can see the secret key.

2. Easier Hacking

Hardcoded secrets can be used to:

- Access databases
- Generate fake JWT tokens
- Impersonate users
- Compromise applications

This can lead to serious security breaches.

3. Difficult Environment Management

Different environments require different configurations.

Example:

Development Database

Testing Database

Production Database

Hardcoding requires changing source code every time.

Using environment variables avoids this problem.

4. Poor Maintainability

Suppose the database password changes.

With hardcoded values:

```
DATABASE_PASSWORD = "old_password"
```

Developers must modify the source code and redeploy the application.

With BaseSettings:

Only the `.env` file needs to be updated.

5. Industry Best Practice

Modern cloud platforms such as:

- AWS
- Azure
- Google Cloud
- Docker
- Kubernetes

recommend storing secrets outside the application code.

This improves security and simplifies deployment.
